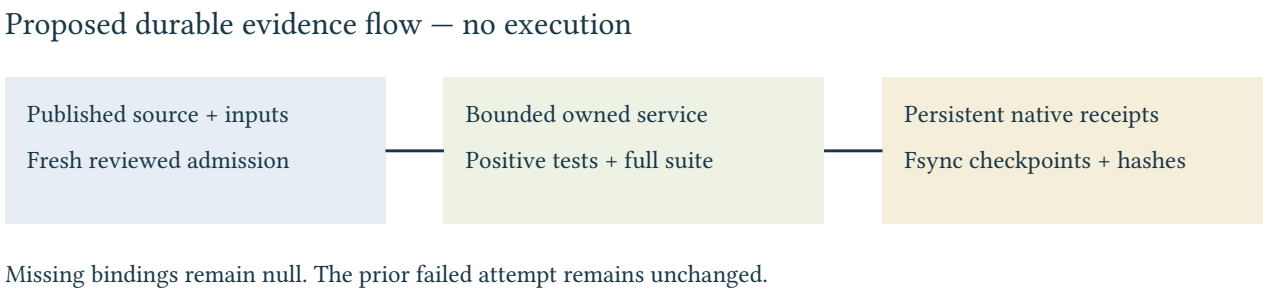
# Durable validation source proposal, v2




Durable root-library validation successor — proposed, unadmitted

This is the v2 draft; the original v1 proposal remains unchanged at commit ae96197a81. T-1598 is source preparation only. It neither reopens T-1566 nor renews its exhausted attempts. No compiler, test, unit, cloud build or experiment is authorized by these files. The previous attempt remains failed with its cause unknown.

Source and scientific scope

source-bindings.json pins observed canonical main commit 29861c9a55f9ae5a33e1e09b3866aa397fbde605 and that revision's root Cargo.toml/Cargo.lock. The canonical pencil source has the same 481f94cc... hash as the historical candidate base. candidate.patch is generated against those exact main bytes and produces candidate 555a5c62...; it adds only the original cfg(test) positive control. Nothing has been applied to canonical source. A future source revision requires a new explicit binding/review, never silently substituting a new HEAD.

Keep default features and the actual Some/None solve_with_progress positive control on the unchanged first p739 fixture, seed 17823097723983109480, scalar 31124, denominator 8, coefficient cap 25 million. Keep the three existing exact guards and the full unfiltered cargo test --release --lib --locked --offline -- --test-threads=1. Zero tests, compile-only success, filtered tests or fake-dispatch tests do not satisfy the required full suite. This source validation does not replace the 1,260-target cohort.

Concrete source changes

durable-worker.py and durable-launcher.py are successors of the saved historical wrappers. durable_io.py adds atomic JSON replacement with file and directory fsync, periodic log fsync and hashed-prefix checkpoints, and a terminal output manifest only after the supervisor verifies the owned cgroup is empty. The producer writes native receipts at startup, containment verification, before each command, after child PID capture, periodically during execution, after each command and at terminal state. Compiler/test stdout and stderr go directly to durable paths; they never require a final copy out of /var/tmp.

The proposal separates temporary workspace/cache from /home/ec2-user/pencil-validation-receipts/<unique-attempt>/. That persistent location must be approved, exist with private ownership, and have its actual mounted filesystem and free capacity recorded before release. Local existence alone is not a remote durability guarantee. A native checkpoint can survive a payload failure; sudden host/storage failure can still lose bytes after the last successful fsync. Running log manifests certify only observed prefixes, not a sealed complete result.

Supervisor receipts preserve argv, unit properties, InvocationID, cgroup, heartbeat identity, systemctl outputs and cleanup observations. The launcher refuses any existing attempt directory or pre-existing unit and only stops a unit with an identical captured InvocationID. If launch observation fails before ownership capture, it records unresolved cleanup and does not kill an unowned unit. The worker waits at most ten seconds for a short-lived supervisor permit tied to its exact InvocationID and lease before input hashing or compilation. The supervisor issues a permit only after ownership capture; every worker guard checks its expiration. Missing/expired permits stop the payload; a blocked uninterruptible syscall still depends on kernel progress and final unit containment. Duplicate durable output directories never become owned, so failure-finally does not overwrite prior receipts. Heartbeat payload fields are task_id, attempt_id, lease_id, fencing_epoch; there is no epoch alias. Fresh coordination ownership is required for execution; T-1598 is not that execution lease.

Proposed resource and retry bounds

One attempt, no automatic retries; 1,800 total wall seconds including input checks, compilation, exact tests, full suite and cleanup. The outer command is timeout --signal=TERM --kill-after=5s 1795s python3 -B durable-launcher.py. The transient unit has 2 aggregate CPUs, 16 GiB memory, no swap, TasksMax 512, OOMPolicy stop, KillMode control-group and a runtime clipped to the remaining absolute deadline. The worker also retains a 16 GiB address-space bound and single Cargo job / test thread. Those latter settings are not substitutes for cgroup enforcement.

Target cache and free-space stop thresholds remain 6 GiB; combined worker logs 256 MiB. These are sampled thresholds, not filesystem quotas, and overshoot must be retained. The v2 draft adds sampled stop thresholds of 64 MiB durable metadata and 384 MiB total durable output, with a 16 MiB limit on each JSON checkpoint. These remain proposed and require review;

cleanup observations may add bounded checkpoint data after a stop threshold trips. Persistent storage free space and mount identity remain release bindings. The outer wall timeout also bounds receipt hashing/sealing; a killed sealing phase remains incomplete, with prior checkpoints retained. Source preparation and artifact transport costs are separately counted, never hidden as zero. Admission rejects booleans and nonfinite dollar values; a hash-bound approved allocation receipt must cover both the finite incremental ceiling and its worst-case estimate within the remaining total allocation. This does not turn delayed AWS billing into exact real-time spend.

A new finite dollar ceiling, absolute UTC deadline, live CPU/RAM admission evidence and allocation are intentionally null. The expired November 4 deadline and historical USD 0.84672 estimate are not reused as authorization. Root is checking funds/live caps separately. No added infrastructure, IAM, cluster privilege or resource pool is proposed.

Required before any execution

1. Materialize a fresh private source tree from the pinned main commit, apply exactly candidate.patch, and verify the candidate source hash. Old /var/tmp paths are gone and must not be reused as if their files survived.
2. Build a new materialized-inputs.json over the complete default-feature root source and path dependencies, recursive include inputs, exact lockfile registry/vendor closure, selected cache artifacts, toolchain and system dependencies. Bind each file's resolved path, bytes and SHA-256; resolve every missing input before any compiler. Preserve source-materialization overhead. No network fallback or broad writable shared cache.
3. Bind prospective unique unit/workspace/durable paths, pinned toolchain, actual storage/cgroup controls, new valid lease fields, finite dollars/deadline and total attempt limit. Independently review worker/launcher failure paths, checkpoint/manifest scope, disk accounting and no-foreign-kill behavior. Ordered exact_tests and all critical fixed recipe limits must match the hash-bound proposal; an empty control list is rejected. Current wrappers are source proposals and have not been syntax-checked, compiled, executed or tested.
4. Commit and push exact source, protocol/configuration, fixture/seeds, complete input manifest and launch command; verify exact GitHub revision and retain its publication receipt before any cloud build or experiment. A local commit or S3-only script is insufficient. The release's admission.json must bind that receipt and every recipe/input file; it is deliberately absent here.
5. Obtain a new explicit coordinator execution release. Publish any resulting attempt, including failures or incomplete receipts, with hashes, costs and native identifiers. A successful validation requires all exact tests, the complete unfiltered suite, unchanged inputs, and owned cleanup; missing evidence stays unknown.

Publication status

This packet is an inspectable source proposal, not a runnable admission. The absent admission.json makes both entrypoints fail before launching any unit or compiler. All historical packets stay untouched. Graphs show the proposed evidence flow rather than unmeasured timing or cost improvement.

Review response

The v1 independent source review found five issues: duplicate-output failure could overwrite earlier receipts; launch observation could lose ownership before cleanup; admission could omit exact tests; a growing log could include unflushed bytes in a reported prefix; and nonfinite/bool dollar ceilings were accepted. The v2 source addresses each explicitly. The log checkpoint fixes the byte count before hashing, reads exactly that prefix, then fsyncs before publishing its prefix digest. No wrapper has been executed, so these are proposed source corrections awaiting independent source review and prospective runtime qualification.

Second source-review corrections

The first v2 read-only review identified concurrent checkpoint tempfile disappearance during size scans and unhandled terminal manifest sealing failure. The size scan now tolerates FileNotFoundError only for the explicit atomic checkpoint temporary filename pattern; all other missing files and I/O errors remain failures. Before sealing, the native receipt records pending_seal. A separate completion.json is created only after owned cleanup and successful final manifest hashing; it binds that manifest digest. A sealing exception records failed/unsealed when storage permits. Consumers must require completion.json with successful seal status and matching manifest, not interpret pending_seal as success. No wrapper execution has occurred.

Root observed newer main commits during publication; this proposal deliberately retains the local observed source revision 29861c9a... rather than silently rebinding it. Fresh GitHub publication verification and any source rebase remain prospective release obligations.

Acceptance conjunction

A completed validation requires all of: the outer timeout command exited zero within its envelope; completion.json reports passed with its UTC/elapsed/deadline fields inside bounds; its manifest hash matches and every manifest member matches; the sealed native receipt has no errors and records payload_status passed; the complete test and post-input verification evidence is present; and owned cgroup cleanup is complete. The pending_seal value of the sealed native receipt is intentional and is only resolved by this full conjunction. A completion marker alone is insufficient. A signal or I/O failure during final marker replacement can leave mixed records; any nonzero outer exit, native failure, missing or mismatched manifest rejects acceptance. The caller must durably retain the actual outer command exit and stdout/stderr at release-bound paths; those caller record paths remain a prospective input binding, not a measured fact.